INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Steganography Analysis, Data Hiding and Passphrase Recovery

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF202

Instructor: Aminu Idris

September 5, 2026

EXECUTIVE SUMMARY

This laboratory report documents the practical execution of digital steganography analysis, evidence preservation, covert payload embedding, hash verification, and automated passphrase recovery. Operating within a Kali Linux forensic workstation environment covert communications were constructed and analyzed using `steghide` and dictionary recovery tools (steggracker).

The primary objective of this laboratory exercise is to demonstrate the principles of steganographic data hiding, least significant bit (LSB) manipulation, and forensic detection. By analyzing original cover media against stego-objects, byte-level alterations and cryptographic hash shifts were verified, establishing a repeatable forensic workflow for identifying and extracting concealed evidence.

LAB ENVIRONMENT & TOOLING INVENTORY

Tool / Dependency	*Primary Function in Forensic Workflow*
Linux (Kali Workstation)	Primary forensic investigation environment (garba@kali).
Wget / Curl	Secure acquisition of authorized laboratory carrier media and remote evidence files.
Steghide	Steganographic embedding and extraction engine utilizing passphrase-based encryption.
StegCracker	Dictionary-based passphrase recovery utility for brute-forcing steganographic objects.
Md5sum	Cryptographic hashing engine for evidence preservation and integrity validation.
ImageMagick (display)	Graphical rendering utility used to verify cover image visual integrity.
Rockyou.txt	Standard forensic wordlist used for controlled passphrase dictionary testing.

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. Steganography vs. Cryptography:

Cryptography scrambles the content of a message so that unauthorized parties cannot read it, but it reveals the existence of the communication.

Steganography conceals the very existence of the message by hiding it inside an innocent-looking cover medium (such as a `.bmp` or `.jpg` image), ensuring an observer does not suspect hidden data is present.

2. Insertion vs. Substitution Techniques (LSB)

Insertion-based steganography appends hidden data outside normal image structures (such as after the file trailer/footer `0xFFD9`).

Substitution-based steganography replaces redundant bits within the cover image itself. Least Significant Bit (LSB) substitution overwrites the lowest-order bit of pixel color values. Because changing the last bit alters color intensity imperceptibly to the human eye, the image appears visually identical while carrying hidden payloads.

SECTION B: PRACTICAL LAB EXECUTION & EVIDENTIARY ANALYSIS

TASK 1: EVIDENCE PRESERVATION & CARRIER ANALYSIS

Part A: Workspace Preparation & Steghide Verification

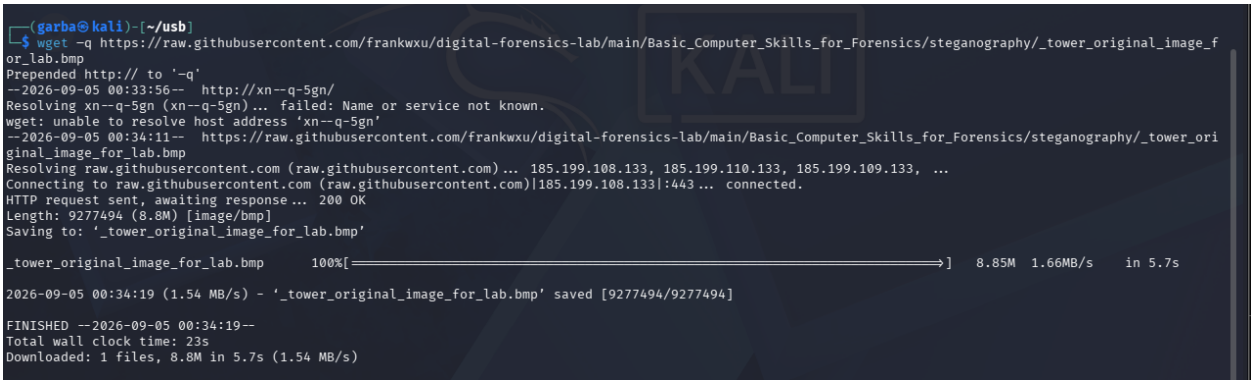
A clean workspace was initialized in `~/usb`. Executing `steghide` without arguments confirmed tool availability and option flags:

steghide

Part B: Carrier Image Download & Visual Inspection

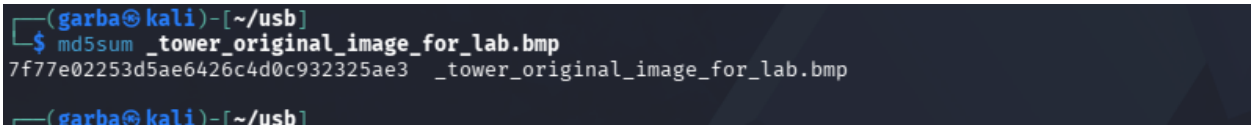
The target cover image was retrieved directly from the repository using wget:

wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp



The carrier image was rendered graphically via ImageMagick to confirm visual integrity:

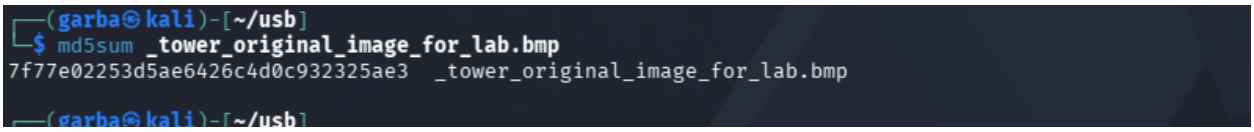
display -resize 400x600 _tower_original_image_for_lab.bmp



Part C: Baseline Hash Calculation

Before performing any modification, the baseline MD5 hash of the original carrier image was recorded:

md5sum _tower_original_image_for_lab.bmp



TASK 2: SECRET EVIDENCE FILE CREATION

A controlled plain text file containing a covert message was generated using echo redirection:

echo "this is a secret file you want to hide" > secret.txt

cat secret.txt

```
┌──(garba㉿kali)-[~/usb]
└─$ echo this is a secret file you want to hide > secret.txt

┌──(garba㉿kali)-[~/usb]
└─$ cat secret.txt
this is a secret file you want to hide
```

Terminal session creating and verifying payload file `secret.txt`.*

TASK 3: STEGANOGRAPHIC EMBEDDING WITH STEGHIDE

Using steghide, the secret text payload was embedded into the carrier image using the lab passphrase 1234

steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234

```
┌──(garba㉿kali)-[~/usb]
└─$ steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234
embedding "secret.txt" in "_tower_original_image_for_lab.bmp"... done

┌──(garba㉿kali)-[~/usb]
└─$ md5sum _tower_original_image_for_lab.bmp
95ca51e0498ca7e98bea06793b852115  _tower_original_image_for_lab.bmp

┌──(garba㉿kali)-[~/usb]
└─$ ls _tower_original_image_for_lab.bmp -l
-rw-rw-r-- 1 garba garba 9277494 Sep  5 00:41 _tower_original_image_for_lab.bmp
```

Post-Embedding Hash Verification

Following embedding, the file's cryptographic hash was recomputed:

md5sum _tower_original_image_for_lab.bmp


```
┌──(garba㉿kali)-[~/usb]
└─$ steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -p 1234
embedding "secret.txt" in "_tower_original_image_for_lab.bmp"... done

┌──(garba㉿kali)-[~/usb]
└─$ md5sum _tower_original_image_for_lab.bmp
95ca51e0498ca7e98bea06793b852115  _tower_original_image_for_lab.bmp

┌──(garba㉿kali)-[~/usb]
└─$ ls _tower_original_image_for_lab.bmp -l
-rw-rw-r-- 1 garba garba 9277494 Sep  5 00:41 _tower_original_image_for_lab.bmp
```


While the visual rendering remained identical to the naked eye, the altered MD5 checksum proves that byte-level data insertion occurred inside the image container.

TASK 4: CONCEALED EVIDENCE EXTRACTION & VERIFICATION

The embedded payload was extracted from the carrier image using the passphrase `1234` and redirected to `xsecret.txt`:

steghide extract -sf _tower_original_image_for_lab.bmp -xf xsecret.txt -p 1234

cat xsecret.txt

The recovered plain text string matched `secret.txt` byte-for-byte, confirming complete payload restoration without data loss.

```
┌──(garba㉿kali)-[~/usb]
└─$ steghide extract -sf _tower_original_image_for_lab.bmp -xf xsecret.txt -p 1234
wrote extracted data to "xsecret.txt".

┌──(garba㉿kali)-[~/usb]
└─$ cat xsecret.txt
this is a secret file you want to hide
```

TASK 5: CONTROLLED PASSPHRASE DICTIONARY TEST

To simulate a scenario where the embedding passphrase is unknown, a dictionary recovery attack was conducted.

Part A: Wordlist Decompression

The system wordlist rockyou.txt.gz was decompressed using sudo gunzip

sudo gunzip /usr/share/wordlists/rockyou.txt.gz

ls /usr/share/wordlists/rockyou.txt

```
┌──(garba㉿kali)-[~/usb]
└─$ ls /usr/share/wordlists
dirb       dnsmap.txt     fern-wifi  legion      nmap.lst        sqlmap.txt  wifite.txt
dirbuster  fasttrack.txt  john.lst   metasploit  rockyou.txt.gz  wfuzz

┌──(garba㉿kali)-[~/usb]
└─$ gunzip /usr/share/wordlists/rockyou.txt.gz

gzip: /usr/share/wordlists/rockyou.txt: Permission denied

┌──(garba㉿kali)-[~/usb]
└─$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
[sudo] password for garba:
Sorry, try again.
[sudo] password for garba:

┌──(garba㉿kali)-[~/usb]
└─$ ls /usr/share/wordlists/rockyou.txt
/usr/share/wordlists/rockyou.txt
```

Decompressing and verifying rockyou.txt within /usr/share/wordlists/.

Part B: Dictionary Brute-Force Execution

stegcracker was executed against the stego image using rockyou.txt:

stegcracker _tower_original_image_for_lab.bmp /usr/share/wordlists/rockyou.txt

Counting lines in wordlist..

Attacking file '_tower_original_image_for_lab.bmp' with wordlist '/usr/share/wordlists/rockyou.txt'..

Successfully cracked file with password: 1234

Tried 2037 passwords

Your file has been written to: _tower_original_image_for_lab.bmp.out

1234

```
┌──(garba㉿kali)-[~/usb]
└─$ stegcracker _tower_original_image_for_lab.bmp /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2026 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

Counting lines in wordlist..
Attacking file '_tower_original_image_for_lab.bmp' with wordlist '/usr/share/wordlists/rockyou.txt
'..
Successfully cracked file with password: 1234
Tried 2037 passwords
Your file has been written to: _tower_original_image_for_lab.bmp.out
1234
```

StegCracker output demonstrating successful passphrase cracking (`1234`) after testing 2,037 entries.

FORENSIC LIMITATIONS & LESSONS LEARNED

1. **Passphrase Dependency:** Steganographic utilities like steghide wrap embedded payloads in encryption (Rijndael-128). Without acquiring or brute-forcing the exact passphrase, payload recovery is mathematically impossible.

2. **Hash Sensitivity:** Modifying pixel bits within a carrier image leaves visual rendering untouched but causes a total hash shift (`MD5` shift from `7f77e022...` to `95ca51e0...`), making cryptographic hashing the primary tool for steganography detection.

3. **Wordlist Efficiency:** Standard dictionary utilities like stegcracker rely heavily on wordlist depth. Complex passphrases missing from standard dictionaries require larger specialized wordlists or high-speed utilities like StegSeek.